

Gabarito — Interpretação de Código

Exercícios 03, 07 e 11

Disciplina: Algoritmos e Programação
Professor: Pedro Bilar Montero

Exercício 03 — Interpretação de Código

Entradas fornecidas: $x = 8$ e $y = 3$

Código analisado

```
1 r1 = x * y + 4;  
2 r2 = x % y;  
3 r3 = (x + y) * 2;  
4 printf("A: %d\n", r1);  
5 printf("B: %d\n", r2);  
6 printf("C: %d\n", r3);
```

Passo a passo

Var.	Cálculo detalhado	Result.
r1	$8 * 3 + 4 \Rightarrow$ mult. tem prioridade $\Rightarrow$ $24 + 4$	28
r2	$8 \% 3 \Rightarrow$ em $8 \div 3$, o quociente é 2 e o resto é 2	2
r3	$(8 + 3) * 2 \Rightarrow$ parênteses primeiro $\Rightarrow$ $11 * 2$	22

Saída na tela:

```
A: 28  
B: 2  
C: 22
```

Pontos de atenção:

- Em $r1 = x * y + 4$, a **multiplicação tem prioridade** sobre a adição — portanto $x * y$ é calculado primeiro.
- Em $r2 = x \% y$, o operador $\%$ retorna o **resto da divisão inteira**: $8 \div 3 = 2$ com resto **2**.
- Em $r3 = (x + y) * 2$, os **parênteses forçam** a adição a acontecer antes da multiplicação.

Exercício 07 — Interpretação de Código

Entradas fornecidas: nota = 6.5 e faltas = 3

Código analisado

```

1 if (faltas > 5) {
2     printf("Reprovado por falta\n");
3 } else if (nota >= 7.0) {
4     printf("Aprovado\n");
5 } else if (nota >= 5.0) {
6     printf("Recuperacao\n");
7 } else {
8     printf("Reprovado\n");
9 }

```

Avaliação das condições

O programa testa as condições **em ordem**, e executa somente o primeiro bloco cujo teste for **verdadeiro**.

#	Condição	Substituição	Resultado	Ação
1	faltas > 5	3 > 5	FALSO	pula
2	nota >= 7.0	6.5 >= 7.0	FALSO	pula
3	nota >= 5.0	6.5 >= 5.0	VERDADEIRO	executa e para

Saída na tela:

```
Recuperacao
```

Pontos de atenção:

- A condição `faltas > 5` é testada **primeiro** — ela funciona como uma *guarda*: se o aluno tiver muitas faltas, nenhuma das outras condições é sequer avaliada.
- Em uma cadeia `if / else if / else`, **após o primeiro bloco verdadeiro ser executado**, os demais são automaticamente ignorados.
- A **ordem** dos `else if` importa: se os testes de nota estivessem invertidos (primeiro `>= 5.0`, depois `>= 7.0`), um aluno com nota 8 seria classificado incorretamente como `Recuperacao`.

Exercício 11 — Interpretação de Código

Entradas fornecidas: nenhuma (variáveis inicializadas no próprio código).

Código analisado

```

1 int i = 1;
2 int soma = 0;
3 while (i <= 5) {
4     if (i % 2 != 0) {
5         soma = soma + i;
6     }
7     i++;
8 }
9 printf("Soma: %d\n", soma);

```

O que o laço faz?

A cada iteração, o programa verifica se i é **ímpar** ($i \% 2 \neq 0$). Se for, soma i ao acumulador. Em seguida, incrementa i .

Tabela de rastreamento

i	$i \leq 5?$	$i \% 2 \neq 0?$	soma	$i++$
1	SIM	SIM (ímpar)	$0 + 1 = 1$	2
2	SIM	NÃO (par)	1 (inalterado)	3
3	SIM	SIM (ímpar)	$1 + 3 = 4$	4
4	SIM	NÃO (par)	4 (inalterado)	5
5	SIM	SIM (ímpar)	$4 + 5 = 9$	6
6	NÃO	—	9 (encerra)	—

Saída na tela:

```
Soma: 9
```

Pontos de atenção:

- O programa calcula $1 + 3 + 5 = 9$, que é a **soma dos números ímpares de 1 a 5**.
- O acumulador **soma** deve ser **inicializado em 0** antes do laço — caso contrário, partiria de um valor imprevisível.
- O teste $i \leq 5$ é verificado **antes** de cada iteração (**while**). Quando i chega a 6, a condição é **falsa** e o laço encerra sem executar o corpo.
- $i++$ é equivalente a $i = i + 1$. Esquecer esse incremento causaria um **laço infinito**.